

REAL-TIME HIGH-FREQUENCY FRAUD & FINANCIAL INTELLIGENCE ENGINE

12-Week Mastery Roadmap • ■30 LPA Fresher Role Preparation

12 Weeks	4 Phases	20+ Technologies	■30 LPA Target
---------------------	---------------------	-----------------------------	---------------------------

Prepared for: Aspiring ML/Data Engineer

Author: Staff Machine Learning Engineer, Top-Tier Fintech

Stack: Kafka • Spark • FastAPI • PostgreSQL/TimescaleDB • XGBoost • Docker •
Prometheus/Grafana

TABLE OF CONTENTS

OVERVIEW & PHILOSOPHY

- Project Vision & Business Context
- System Architecture Diagram
- Technology Stack & Rationale
- GitHub Project Structure

PHASE 1 — Foundations & System Design (Weeks 1–2)

- Environment Setup & Docker Fundamentals
- Simulating Realistic Imbalanced Fraud Data
- PostgreSQL & TimescaleDB Installation
- Architecture Deep-Dive & Design Decisions
- Mastery Challenge #1

PHASE 2 — The Data Engine (Weeks 3–6)

- Apache Kafka: Concepts to Production
- PySpark Streaming & Mini-Batch Processing
- TimescaleDB: Velocity Feature Engineering
- Complex Window Functions & CTEs
- Handling Late-Arriving Data
- Mastery Challenge #2

PHASE 3 — The Brain (Weeks 7–9)

- XGBoost & Isolation Forest Model Training
- Handling Class Imbalance (SMOTE, Cost-Sensitive)
- ONNX Model Export & Quantization
- FastAPI Inference Service (Sub-20ms)
- Async Processing & Batched Inference
- Mastery Challenge #3

PHASE 4 — The Production Wall (Weeks 10–12)

- Docker Compose Full-Stack Deployment
- CI/CD with GitHub Actions
- Prometheus Metrics & Grafana Dashboards
- Data Drift Detection & Automated Alerts
- Interview Preparation & Portfolio Strategy
- Mastery Challenge #4

APPENDICES

- A: Essential Documentation Links
- B: Recommended Learning Resources
- C: Common Debugging Patterns

PROJECT OVERVIEW & PHILOSOPHY

Project Vision & Business Context

Financial fraud costs the global economy over **\$485 billion annually** (PwC, 2024). Every millisecond of detection latency translates directly into financial loss. This project builds an end-to-end, production-grade, real-time fraud detection system — the exact class of system operated by teams at Razorpay, PhonePe, Paytm, and CRED. By completing this roadmap, you will have a portfolio artifact that demonstrates all three pillars required for a **■30 LPA fresher ML/Data Engineering role: System Architecture, Data Engineering, and MLOps.**

■ ENGINEERING PHILOSOPHY

Build first, understand deeper. Every concept in this roadmap is introduced through hands-on code. You will make it work, then make it right, then make it fast. This mirrors exactly how senior engineers operate on day one in a new codebase.

System Architecture Overview

The system follows a Lambda-inspired streaming architecture with four primary layers:

Layer	Technology	Responsibility	Target SLA
Ingestion	Apache Kafka	Ingest 10K+ TPS transaction stream	< 5ms publish
Processing	PySpark Streaming	Mini-batch feature aggregation, drift detection	< 500ms batch
Feature Store	TimescaleDB (PostgreSQL)	Velocity features, historical stats, lookups	< 10ms query
Inference	FastAPI + ONNX	Model scoring, async batching, response	< 20ms p99
Observability	Prometheus + Grafana	Metrics, drift alerts, dashboard	Real-time
Orchestration	Docker Compose	Full-stack deployment, service mesh	One-command up

Technology Stack & Rationale

Technology	Version	Why This Choice	Learn First At
Python	3.11+	Universal ML/DE language; asyncio for latency	python.org/docs
Apache Kafka	3.6	Industry-standard event streaming; used at every major fintech	kafka.apache.org/documentation
Apache Spark	3.5	Handles TB-scale; Structured Streaming for mini-batches	spark.apache.org/docs
PostgreSQL	16	ACID compliance; powerful window functions for features	postgresql.org/docs
TimescaleDB	2.14	Time-series extension on PG; hypertables for velocity features	timescale.com
FastAPI	0.111	Async-native; automatic OpenAPI docs; ASGI performance	fastapi.tiangolo.com
XGBoost	2.0	Gold standard gradient boosting; sklearn API; ONNX export	xgboost.readthedocs.io

Technology	Version	Why This Choice	Learn First At
ONNX Runtime	1.18	Hardware-optimized inference; quantization for speed	onnxruntime.ai
Docker	26	Reproducibility; mirrors every production environment	docs.docker.com
Prometheus	2.51	Pull-based metrics standard; time-series monitoring	prometheus.io/docs
Grafana	10	Best-in-class dashboards; alert routing	grafana.com/docs

GitHub Project Structure

Maintaining a clean, professional repository structure signals engineering maturity to recruiters. Use the following layout from day one:

```

fraud-intelligence-engine/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml           # Lint + Test on every PR
│   │   └── cd.yml           # Build & push Docker images on merge
│   └── data/
│       ├── generators/
│       │   ├── transaction_simulator.py  # Phase 1: Data simulation
│       │   └── fraud_injector.py         # Phase 1: Fraud pattern injection
│       ├── seeds/                      # Static reference data
│       └── ingestion/
│           ├── kafka_producer.py        # Phase 2: Kafka producer
│           ├── kafka_consumer.py        # Phase 2: Consumer for Spark
│           └── processing/
│               ├── spark_streaming.py    # Phase 2: PySpark streaming job
│               ├── feature_engineering.py # Phase 2: Feature computation
│               └── drift_detector.py     # Phase 4: Data drift detection
│       ├── feature_store/
│       │   ├── migrations/
│       │   │   ├── 001_init_schema.sql   # Phase 2: TimescaleDB schema
│       │   │   └── velocity_queries.sql  # Phase 2: Window function CTEs
│       │   └── feature_server.py         # Phase 3: Feature retrieval API
│       └── model/
│           ├── train.py                 # Phase 3: XGBoost training pipeline
│           ├── evaluate.py              # Phase 3: Precision/Recall/F1/AUC
│           ├── export_onnx.py           # Phase 3: ONNX conversion + quantize
│           └── artifacts/               # Saved models (.onnx, .json)
│       ├── inference/
│       │   ├── main.py                 # Phase 3: FastAPI app entry point
│       │   ├── schemas.py              # Pydantic request/response models
│       │   ├── predictor.py            # ONNX Runtime inference engine
│       │   └── middleware.py           # Latency tracking, auth
│       └── monitoring/
│           ├── prometheus.yml          # Scrape config
│           └── grafana/
│               ├── dashboards/
│               │   ├── model_performance.json
│               │   └── data_drift.json
│               └── provisioning/
│       ├── docker/
│       │   ├── Dockerfile.inference    # Multi-stage FastAPI image
│       │   ├── Dockerfile.spark        # PySpark image
│       │   └── Dockerfile.producer     # Kafka producer image
│       ├── tests/
│       │   ├── unit/                  # Pytest unit tests
│       │   ├── integration/           # End-to-end pipeline tests
│       │   └── load/                  # Locust load tests for < 20ms SLA
│       ├── notebooks/                # Jupyter EDA & experimentation
│       ├── docker-compose.yml         # Full-stack local deployment
│       ├── requirements.txt
│       ├── pyproject.toml             # Ruff linter + Black formatter config
│       └── README.md                  # Architecture diagram + setup guide

```

■ WEEK 1 — Environment & Data

1.1 Development Environment Setup

Before writing a single line of ML code, you must establish a fully reproducible, containerized development environment. This is non-negotiable in production engineering.

- **Docker Desktop 26+** — Containerization runtime (docs.docker.com/get-docker)
- **Python 3.11+** — Core language runtime (python.org/downloads)
- **Git 2.44+** — Version control (git-scm.com)
- **VS Code + Extensions** — Python, Docker, PostgreSQL extensions (code.visualstudio.com)
- **DBeaver Community** — SQL client for TimescaleDB (dbeaver.io)

Initial Docker Compose skeleton — save as `docker-compose.yml`:

YAML

```
version: '3.9'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    depends_on: [zookeeper]
    ports:
      - '9092:9092'
    environment:
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  timescaledb:
    image: timescale/timescaledb:latest-pg16
    ports:
      - '5432:5432'
    environment:
      POSTGRES_PASSWORD: fraud_engine_secret
      POSTGRES_DB: fraud_db
    volumes:
      - tsdb_data:/var/lib/postgresql/data

  prometheus:
    image: prom/prometheus:v2.51.0
    ports: ['9090:9090']
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml

  grafana:
    image: grafana/grafana:10.3.3
    ports: ['3000:3000']
    environment:
      GF_SECURITY_ADMIN_PASSWORD: admin

volumes:
  tsdb_data:
```

■ LEARN THIS FIRST

Docker concepts to master before proceeding: images vs containers, volumes, networks, depends_on, and health checks. Read: docs.docker.com/get-started/overview/ Estimated time: 3-4 hours including hands-on exercises.

1.2 Simulating Realistic Imbalanced Fraud Data

Real fraud datasets are confidential. We simulate one with statistically correct properties: **0.1%–0.5% fraud rate** (severe class imbalance), realistic transaction amounts following a log-normal distribution, temporal patterns (weekday vs weekend), and distinct fraud archetypes (card testing, account takeover, bust-out fraud).

transaction_simulator.py — Core simulation engine:

```

import numpy as np
import pandas as pd
from faker import Faker
from datetime import datetime, timedelta
import json, uuid, random

fake = Faker('en_IN') # Indian locale for realistic data
rng = np.random.default_rng(seed=42) # Reproducible

FRAUD_ARCHETYPES = {
    'card_testing': {'amount_range': (1, 50), 'velocity': 'high', 'weight': 0.35},
    'account_takeover': {'amount_range': (5000, 50000), 'velocity': 'med', 'weight': 0.40},
    'bust_out': {'amount_range': (1000, 10000), 'velocity': 'low', 'weight': 0.25},
}

def generate_transaction(user_id: str, is_fraud: bool = False) -> dict:
    base_amount = float(rng.lognormal(mean=6.5, sigma=1.2)) # Log-normal dist
    if is_fraud:
        archetype = random.choices(
            list(FRAUD_ARCHETYPES.keys()),
            weights=[v['weight'] for v in FRAUD_ARCHETYPES.values()]
        )[0]
        lo, hi = FRAUD_ARCHETYPES[archetype]['amount_range']
        amount = round(random.uniform(lo, hi), 2)
    else:
        amount = round(base_amount, 2)

    return {
        'transaction_id': str(uuid.uuid4()),
        'user_id': user_id,
        'timestamp': datetime.utcnow().isoformat() + 'Z',
        'amount': amount,
        'merchant_id': fake.bothify(text='MER-####??'),
        'merchant_cat': random.choice(['grocery', 'travel', 'electronics',
                                       'restaurant', 'utility', 'crypto']),
        'country': random.choices(['IN', 'US', 'SG', 'AE'],
                                  weights=[0.70, 0.15, 0.10, 0.05])[0],
        'device_id': fake.md5()[:16],
        'ip_address': fake.ipv4(),
        'is_fraud': int(is_fraud), # Label (hidden at inference time)
    }

def generate_dataset(n_transactions: int = 100_000,
                    fraud_rate: float = 0.003) -> pd.DataFrame:
    user_pool = [fake.uuid4() for _ in range(10_000)]
    fraud_count = int(n_transactions * fraud_rate)
    legit_count = n_transactions - fraud_count

    records = (
        [generate_transaction(random.choice(user_pool), False) for _ in range(legit_count)]
        + [generate_transaction(random.choice(user_pool), True) for _ in range(fraud_count)]
    )
    df = pd.DataFrame(records).sample(frac=1, random_state=42).reset_index(drop=True)
    print(f'Generated {len(df):,} transactions | Fraud rate: {df.is_fraud.mean():.3%}')
    return df

```

■ BEGINNER CHECKPOINT

Before moving forward, run this script and verify: (1) The DataFrame shape is (100000, 11). (2) `df.is_fraud.value_counts()` shows ~300 fraudulent transactions. (3) `df.amount.describe()` shows a realistic distribution with mean ~800-1200 INR. If not, debug your random seed and distribution parameters.

■ WEEK 2 — Architecture & Database

1.3 PostgreSQL & TimescaleDB Schema Design

TimescaleDB extends PostgreSQL with hypertables — automatically partitioned by time. This makes time-range queries on billions of rows 10–100x faster than standard PostgreSQL. Understanding this is critical for velocity feature engineering in Phase 2.

SQL

```
-- 001_init_schema.sql
-- Run: psql -h localhost -U postgres -d fraud_db -f 001_init_schema.sql

CREATE EXTENSION IF NOT EXISTS timescaledb;

-- Core transaction table (will become a hypertable)
CREATE TABLE transactions (
  transaction_id  UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id        TEXT          NOT NULL,
  timestamp      TIMESTAMPTZ  NOT NULL,
  amount         NUMERIC(12, 2) NOT NULL,
  merchant_id    TEXT,
  merchant_cat   TEXT,
  country        CHAR(2),
  device_id      TEXT,
  ip_address     INET,
  is_fraud       SMALLINT     DEFAULT 0
);

-- Convert to hypertable partitioned by hour
SELECT create_hypertable('transactions', 'timestamp',
                        chunk_time_interval => INTERVAL '1 hour');

-- Continuous aggregate for hourly velocity stats
CREATE MATERIALIZED VIEW user_hourly_stats
WITH (timescaledb.continuous) AS
  SELECT
    user_id,
    time_bucket('1 hour', timestamp) AS bucket,
    COUNT(*)                        AS txn_count,
    SUM(amount)                     AS total_amount,
    AVG(amount)                     AS avg_amount
  FROM transactions
  GROUP BY user_id, bucket
WITH NO DATA;

-- Auto-refresh every 30 seconds
SELECT add_continuous_aggregate_policy('user_hourly_stats',
  start_offset => INTERVAL '2 hours',
  end_offset   => INTERVAL '30 seconds',
  schedule_interval => INTERVAL '30 seconds');

-- Indexes for fast feature lookups
CREATE INDEX idx_txn_user_time ON transactions (user_id, timestamp DESC);
CREATE INDEX idx_txn_device   ON transactions (device_id, timestamp DESC);
CREATE INDEX idx_txn_ip       ON transactions (ip_address, timestamp DESC);
```

1.4 Architecture Design Decisions

Every architectural choice involves trade-offs. Interviewers at **■30 LPA** levels will probe these. Here is the rationale for this system's key decisions:

■ Why Kafka over direct API calls?

Kafka decouples producers from consumers, provides replay capability (critical for debugging fraud cases), and handles traffic spikes via buffering. At 10K TPS, synchronous API calls would cascade failures.

■ Why TimescaleDB over Redis for the Feature Store?

Redis is faster but loses data on restart (without persistence config). TimescaleDB gives us SQL expressiveness for complex window queries, ACID guarantees, and continuous aggregates that auto-compute hourly stats. For < 10ms feature lookups, both work; TimescaleDB wins on query complexity.

■ Why ONNX over serving the native XGBoost model?

ONNX Runtime applies hardware-specific optimizations (SIMD, graph fusion) automatically. In benchmarks, ONNX-quantized XGBoost is 2–4x faster than native Python inference. This is the difference between 40ms and 18ms p99 latency.

■ Why FastAPI over Flask?

FastAPI is ASGI-native, meaning it handles async I/O without blocking. One sync endpoint in Flask under heavy load blocks the entire process. FastAPI's async handlers allow concurrent feature store queries during inference.

■ MASTERY CHALLENGE #1

Extend the transaction simulator to model a realistic **card testing attack**: one user_id making 50+ transactions under ■50 each within 10 minutes, then one large transaction of ■40,000. Write a SQL query against TimescaleDB that flags this pattern using a window function. Verify the hypertable is partitioning correctly by running EXPLAIN ANALYZE on your query.

■ WEEKS 3–4 — Kafka & Spark

2.1 Apache Kafka: From Concepts to Production

Kafka is a distributed, fault-tolerant, high-throughput event streaming platform. Before writing code, internalize these five core concepts:

Concept	Definition	Analogy
Topic	Named stream of records (e.g., 'transactions')	A database table for events
Partition	Ordered, immutable log within a topic	A shard / horizontal slice
Offset	Unique ID for each record within a partition	Row number in the shard
Producer	Client that publishes records to topics	INSERT INTO transactions
Consumer Group	Set of consumers that jointly read a topic	Distributed SELECT workers
Broker	Kafka server that stores partitions	Database server node

Kafka Producer — Streaming Transactions:

```
# ingestion/kafka_producer.py
from confluent_kafka import Producer
from data.generators.transaction_simulator import generate_transaction
import json, time, random, logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

KAFKA_CONFIG = {
    'bootstrap.servers': 'localhost:9092',
    'client.id': 'fraud-producer',
    # Performance tuning – critical for high-throughput
    'linger.ms': 5, # Batch records for 5ms before sending
    'batch.size': 16384, # 16KB batches
    'compression.type': 'snappy', # ~40% size reduction
    'acks': '1', # Leader ack only (throughput vs durability trade-off)
}

def delivery_report(err, msg):
    if err:
        logger.error(f'Message delivery failed: {err}')

def stream_transactions(tps: int = 1000, duration_seconds: int = 300):
    producer = Producer(KAFKA_CONFIG)
    user_pool = [f'user_{i:05d}' for i in range(10_000)]
    interval = 1.0 / tps
    start = time.monotonic()
    count = 0

    while time.monotonic() - start < duration_seconds:
        is_fraud = random.random() < 0.003 # 0.3% fraud rate
        txn = generate_transaction(random.choice(user_pool), is_fraud)
        producer.produce(
            topic = 'transactions',
            key = txn['user_id'].encode(), # Key = user_id for partitioning
            value = json.dumps(txn).encode(),
            callback = delivery_report
        )
        producer.poll(0) # Non-blocking delivery callback check
        count += 1
        time.sleep(interval)

    producer.flush() # Ensure all buffered messages are sent
    logger.info(f'Streamed {count:,} transactions in {duration_seconds}s')

if __name__ == '__main__':
    stream_transactions(tps=500, duration_seconds=600)
```

2.2 PySpark Streaming & Mini-Batch Processing

PySpark Structured Streaming reads from Kafka in micro-batches, applies transformations, and writes enriched features to TimescaleDB. The key concept is the **trigger interval** — how often Spark processes a new batch. We use 10-second micro-batches.

PySpark Structured Streaming — Part 1: Setup & Kafka Read:

[illegible]

PySpark Structured Streaming — Part 2: Windowed Aggregation & Write:

[illegible]

■ WEEKS 5-6 — SQL Feature Engineering

2.3 TimescaleDB: Velocity Feature Engineering with SQL

This is the most technically demanding section for interviews. **Velocity features** measure how fast a user's behavior is changing relative to their own historical baseline. A user making 30 transactions in 10 minutes when their historical average is 3 per day is a critical fraud signal.

Complex Window Functions & CTEs — Part 1: Recent Activity & Historical Baseline:

```
-- velocity_queries.sql (Part 1/2)
-- PURPOSE: Compute velocity features comparing current vs historical behavior.
```

Velocity SQL — Part 2: Window Rank, Velocity Ratio & Final Output:

```
-- velocity_queries.sql (Part 2/2 — continuation of WITH clause above)

-- ■■ CTE 3: Sliding Window Rank ████████████████████████████████████████████████████████████████████████████████
window_velocity AS (
    SELECT
        user_id, timestamp, amount,
        COUNT(*) OVER (
            PARTITION BY user_id ORDER BY timestamp
            RANGE BETWEEN INTERVAL '1 hour' PRECEDING AND CURRENT ROW
        )
        AS txn_count_1h,
        COUNT(*) OVER (
            PARTITION BY user_id ORDER BY timestamp
            RANGE BETWEEN INTERVAL '24 hours' PRECEDING AND CURRENT ROW
        )
        AS txn_count_24h,
        -- Z-score: how anomalous is this transaction amount for this user?
        (amount - AVG(amount) OVER (PARTITION BY user_id)) /
        NULLIF(STDDEV(amount) OVER (PARTITION BY user_id), 0)
        AS amount_zscore,
        -- Inter-transaction time (low gap = high velocity attack signal)
        timestamp - LAG(timestamp) OVER (
            PARTITION BY user_id ORDER BY timestamp
        )
        AS time_since_last_txn
    FROM transactions
    WHERE user_id = $1 AND timestamp >= NOW() - INTERVAL '25 hours'
),

-- ■■ CTE 4: Velocity Ratio Calculation ████████████████████████████████████████████████████████████████████████████████
velocity_ratio AS (
    SELECT
        r.user_id,
        r.txn_count_10m, r.total_amount_10m,
        r.unique_merchants_10m, r.unique_countries_10m,
        h.avg_daily_txn_count, h.avg_daily_amount,
        -- KEY SIGNAL: 10-min rate vs daily average (144 = 10-min slots per day)
        CASE WHEN COALESCE(h.avg_daily_txn_count, 0) = 0 THEN NULL
             ELSE (r.txn_count_10m * 144.0) / h.avg_daily_txn_count
        END
        AS velocity_ratio,
        CASE WHEN COALESCE(h.avg_daily_amount, 0) = 0 THEN NULL
             ELSE r.total_amount_10m / h.avg_daily_amount
        END
        AS amount_spike_ratio
    FROM recent_activity r
    LEFT JOIN historical_baseline h USING (user_id)
)
SELECT
    vr.*,
    (velocity_ratio > 10)::int AS high_velocity_flag,
    (amount_spike_ratio > 5)::int AS amount_spike_flag
FROM velocity_ratio vr;
```

2.4 Handling Late-Arriving Data (Critical Concept)

In distributed systems, events arrive out of order. A transaction processed at T+5 minutes might have actually occurred at T-3 minutes (network delay, mobile app sync, retries). If you ignore late data, your 10-minute velocity window is wrong.

■ ■ LATE DATA PROBLEM

Example: User makes 8 transactions between 14:00-14:10. Three of them arrive at the Kafka broker at 14:14 due to a flaky mobile connection. Without watermarking, your 10-minute window at 14:10 only sees 5 transactions and misses the fraud signal. Watermarks define how late data can arrive before being dropped.

```
# In PySpark Structured Streaming, watermarks tell Spark:
# 'Allow events up to 2 minutes late. After that, drop them.'

# The .withWatermark('event_ts', '2 minutes') call sets this.
# Spark then keeps state (the window aggregation) for:
#   window_duration + watermark_delay = 10 min + 2 min = 12 minutes

# TRADE-OFF TABLE:
# Watermark = 0s → No late data handled. Fast state cleanup. Wrong results.
# Watermark = 2m → Good balance. 2-min delay tolerance. ~40% of late events caught.
# Watermark = 10m → High tolerance. Slow. Lots of memory for state. Latency impact.

# IMPLEMENTATION: Already in spark_streaming.py (see above)
# .withWatermark('event_ts', '2 minutes')

# MONITORING: Check if events are arriving late
# query.lastProgress['durationMs'] shows processing time per batch
# query.lastProgress['eventTime']['watermark'] shows current watermark

import json
def monitor_streaming_query(query):
    progress = query.lastProgress
    if progress:
        print(f"Batch: {progress['batchId']}")
        print(f"Input rows: {progress['numInputRows'];}")
        print(f"Processing time: {progress['durationMs']['triggerExecution']}ms")
        print(f"Watermark: {progress['eventTime'].get('watermark', 'N/A')}")
```

■ MASTERY CHALLENGE #2

1. Extend the Spark streaming job to detect **impossible travel**: the same `user_id` making a transaction in India and then the UAE within 30 minutes. Write the PySpark logic using a sessionization window. 2. Write a TimescaleDB continuous aggregate that pre-computes the **7-day rolling average transaction count per user** and refreshes every 5 minutes. 3. Benchmark your velocity SQL query using EXPLAIN ANALYZE — the planner should show an Index Scan on `idx_txn_user_time`, not a Seq Scan. If it shows Seq Scan, investigate why and fix it.

■ WEEK 7 — Model Training

3.1 XGBoost Training Pipeline for Fraud Detection

XGBoost is the workhorse of fraud detection in production. It handles mixed feature types natively, is interpretable via SHAP values, and exports cleanly to ONNX. We pair it with Isolation Forest for unsupervised anomaly scoring as a complementary signal.

Feature Engineering for the Model:

PYTHON

```
# model/train.py
import pandas as pd
import numpy as np
import xgboost as xgb
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (roc_auc_score, precision_recall_curve,
                             average_precision_score, classification_report)
from imblearn.over_sampling import SMOTE
import shap, joblib

def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    df['timestamp'] = pd.to_datetime(df['timestamp'])

    # Temporal features
    df['hour_of_day'] = df['timestamp'].dt.hour
    df['day_of_week'] = df['timestamp'].dt.dayofweek
    df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
    df['is_night'] = df['hour_of_day'].between(22, 6).astype(int)

    # Amount features
    df['log_amount'] = np.log1p(df['amount'])
    df['amount_rounded'] = (df['amount'] % 1 == 0).astype(int) # Round amounts = card testing
    df['is_micro_txn'] = (df['amount'] < 10).astype(int)

    # Encode categoricals
    le = LabelEncoder()
    df['merchant_cat_enc'] = le.fit_transform(df['merchant_cat'].fillna('unknown'))
    df['country_enc'] = le.fit_transform(df['country'].fillna('XX'))

    # Velocity features (pre-computed via SQL, joined here)
    # These columns should already exist after SQL feature store join
    velocity_cols = ['txn_count_10m', 'total_amount_10m', 'velocity_ratio',
                    'amount_spike_ratio', 'unique_merchants_10m',
                    'unique_countries_10m', 'txn_count_1h']
    for col in velocity_cols:
        df[col] = df.get(col, pd.Series(np.zeros(len(df))))

    return df

FEATURE_COLS = [
    'log_amount', 'amount_rounded', 'is_micro_txn',
    'hour_of_day', 'day_of_week', 'is_weekend', 'is_night',
    'merchant_cat_enc', 'country_enc',
    'txn_count_10m', 'total_amount_10m', 'velocity_ratio',
    'amount_spike_ratio', 'unique_merchants_10m',
    'unique_countries_10m', 'txn_count_1h',
]
```

Handling Class Imbalance — The Critical Challenge:

With 0.3% fraud rate, a model that predicts 'not fraud' for every transaction achieves 99.7% accuracy but is completely useless. You must use one or more of these strategies:

Strategy	How It Works	When to Use	Code
scale_pos_weight	XGBoost native: ratio of negatives to positives as baseline	Always as baseline	scale_pos_weight=333
SMOTE	Synthetic minority oversampling via KNN interpolation	Small dataset < 1M rows	SMOTE(sampling_strate
Cost-Sensitive Loss	Penalize false negatives more in loss function	When FN cost >> FP cost	xgb param: eval_metric=
Threshold Tuning	Lower decision threshold from 0.5 to 0.1-0.2	High % in fraud; tune to business	threshold = 0.15

PYTHON

```
def train_xgboost(df: pd.DataFrame):
    df = engineer_features(df)
    X = df[FEATURE_COLS].fillna(0)
    y = df['is_fraud']

    fraud_count = y.sum()
    legit_count = len(y) - fraud_count
    pos_weight = legit_count / fraud_count # e.g. 333 for 0.3% rate

    # Apply SMOTE only on training fold
    smote = SMOTE(sampling_strategy=0.1, random_state=42, k_neighbors=5)

    skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    oof_preds = np.zeros(len(X))

    xgb_params = {
        'objective': 'binary:logistic',
        'eval_metric': 'aucpr', # Area under PR curve – right metric for fraud
        'scale_pos_weight': pos_weight,
        'max_depth': 6,
        'learning_rate': 0.05,
        'n_estimators': 500,
        'min_child_weight': 5, # Prevents overfitting on fraud minority
        'subsample': 0.8,
        'colsample_bytree': 0.8,
        'tree_method': 'hist', # Faster for large datasets
        'device': 'cpu', # Change to 'cuda' if GPU available
        'early_stopping_rounds': 30,
        'random_state': 42,
    }

    models = []
    for fold, (train_idx, val_idx) in enumerate(skf.split(X, y)):
        X_tr, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_tr, y_val = y.iloc[train_idx], y.iloc[val_idx]

        X_tr_sm, y_tr_sm = smote.fit_resample(X_tr, y_tr) # SMOTE on train only

        model = xgb.XGBClassifier(**xgb_params)
        model.fit(X_tr_sm, y_tr_sm,
                  eval_set=[(X_val, y_val)],
                  verbose=50)

        oof_preds[val_idx] = model.predict_proba(X_val)[ :, 1]
        models.append(model)
        auprc = average_precision_score(y_val, oof_preds[val_idx])
        print(f'Fold {fold+1} AUPRC: {auprc:.4f}')

    overall_auprc = average_precision_score(y, oof_preds)
    print(f'Overall CV AUPRC: {overall_auprc:.4f}')
    return models[-1] # Return last fold model (or ensemble)
```

3.2 ONNX Model Export & Quantization

ONNX (Open Neural Network Exchange) is a hardware-agnostic model format. ONNX Runtime applies graph-level optimizations and hardware-specific execution providers (Intel MKL-DNN, NVIDIA CUDA, ARM) automatically. Combined with **INT8 quantization**, we reduce model size by ~75% and improve inference speed by 2–4x with less than 1% accuracy loss.

```

# model/export_onnx.py
import xgboost as xgb
import onnxmltools
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
import onnxruntime as rt
from onnxruntime.quantization import quantize_dynamic, QuantType
import numpy as np, time

N_FEATURES = 17 # Must match FEATURE_COLS length in train.py

def export_to_onnx(model: xgb.XGBClassifier, output_path: str):
    '''Export trained XGBoost model to ONNX format.'''
    initial_type = [('float_input', FloatTensorType([None, N_FEATURES]))]
    onnx_model = convert_sklearn(model, initial_types=initial_type,
                                target_opset=18)
    with open(output_path, 'wb') as f:
        f.write(onnx_model.SerializeToString())
    print(f'Exported ONNX model to {output_path}')
    return output_path

def quantize_model(onnx_path: str, quantized_path: str):
    '''Apply INT8 dynamic quantization for faster CPU inference.'''
    quantize_dynamic(
        model_input = onnx_path,
        model_output = quantized_path,
        weight_type = QuantType.QInt8, # INT8 weights
    )
    print(f'Quantized model saved to {quantized_path}')

def benchmark_inference(onnx_path: str, n_runs: int = 1000):
    '''Measure p50, p95, p99 latency for a single prediction.'''
    sess = rt.InferenceSession(onnx_path,
                               providers=['CPUExecutionProvider'])
    input_name = sess.get_inputs()[0].name
    dummy_input = np.random.randn(1, N_FEATURES).astype(np.float32)

    # Warm-up runs (JIT compilation, cache warming)
    for _ in range(20):
        sess.run(None, {input_name: dummy_input})

    latencies = []
    for _ in range(n_runs):
        t0 = time.perf_counter()
        sess.run(None, {input_name: dummy_input})
        latencies.append((time.perf_counter() - t0) * 1000) # ms

    latencies.sort()
    print(f'Inference latency (n={n_runs}):',
          f'p50={latencies[int(n_runs*0.50)]:.2f}ms',
          f'p95={latencies[int(n_runs*0.95)]:.2f}ms',
          f'p99={latencies[int(n_runs*0.99)]:.2f}ms')

if __name__ == '__main__':
    import joblib
    model = joblib.load('model/artifacts/xgb_model.pkl')
    export_to_onnx(model, 'model/artifacts/model.onnx')
    quantize_model('model/artifacts/model.onnx',
                  'model/artifacts/model_quantized.onnx')
    print('=== Original ONNX ===')
    benchmark_inference('model/artifacts/model.onnx')
    print('=== Quantized ONNX ===')
    benchmark_inference('model/artifacts/model_quantized.onnx')

```

■ WEEK 9 — FastAPI Inference Service

3.3 FastAPI Inference Service — Sub-20ms Latency

The inference service receives a transaction, fetches velocity features from TimescaleDB asynchronously, runs ONNX inference, and returns a fraud probability — all within 20ms. Achieving this requires: async DB queries, connection pooling, model pre-loading, and batched inference.

FastAPI App — Part 1: Setup, Metrics & Lifespan:

PYTHON

[illegible]

FastAPI App — Part 2: Scoring Endpoint:

```
# inference/main.py (continued)

@app.post('/v1/score', response_model=FraudResponse)
async def score_transaction(request: TransactionRequest):
    t_start = time.perf_counter()
    REQUESTS_TOTAL.inc()

    # Step 1: Async feature retrieval from TimescaleDB (~6-8ms)
    features = await app.state.extractor.get_features(
        user_id = request.user_id,
        amount = request.amount,
        merchant_cat = request.merchant_cat,
        country = request.country,
        timestamp = request.timestamp,
    )

    # Step 2: ONNX inference (~1-3ms after INT8 quantization)
    fraud_prob = app.state.predictor.predict(features)

    latency_ms = (time.perf_counter() - t_start) * 1000
    INFERENCE_LATENCY.observe(latency_ms)
    FRAUD_SCORE_HIST.observe(fraud_prob)

    if fraud_prob > 0.7:
        FRAUD_ALERTS.inc()

    return FraudResponse(
        transaction_id = request.transaction_id,
        fraud_probability = round(fraud_prob, 4),
        risk_level = 'HIGH' if fraud_prob > 0.7 else
                    'MEDIUM' if fraud_prob > 0.3 else 'LOW',
        latency_ms = round(latency_ms, 2),
        features_used = features.feature_names,
    )

@app.get('/health')
async def health_check():
    return {'status': 'ok', 'model': 'loaded'}
```

■ LATENCY BUDGET BREAKDOWN

Total budget: 20ms. Allocation: Feature store query (asyncpg): 6-8ms. Feature vector construction: 0.5ms. ONNX Runtime inference: 1-3ms. FastAPI serialization + network: 2-4ms. Total: 9.5-15.5ms. Remaining headroom: 4.5-10.5ms. This is your P99 safety margin.

■ MASTERY CHALLENGE #3

1. Add an **Isolation Forest** as a second model. Its anomaly score (-1 to 1) should be a feature fed into the XGBoost model (stacking). 2. Write a load test using **Locust** that simulates 200 concurrent users hitting the `/v1/score` endpoint. Prove that P99 latency stays below 20ms. If it doesn't, profile using **py-spy** and identify the bottleneck. 3. Implement **SHAP explanations** on the XGBoost model and add a `/v1/explain` endpoint that returns the top-3 features driving the fraud score for any given transaction.

THE PRODUCTION WALL

Weeks 10–12 • Docker • CI/CD • Monitoring • Drift Detection

■ WEEK 10 — Docker Production Deployment

4.1 Multi-Stage Docker Build for Inference Service

Multi-stage builds are the production standard. Stage 1 (builder) compiles dependencies in a full Python image. Stage 2 (runtime) copies only the compiled artifacts into a slim image. Result: 2.1GB → 280MB image size.

```
# docker/Dockerfile.inference

# ■■ Stage 1: Build dependencies
FROM python:3.11-slim AS builder
WORKDIR /build

RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential gcc && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ■■ Stage 2: Lean runtime image
FROM python:3.11-slim AS runtime
WORKDIR /app

# Non-root user for security
RUN useradd -m -u 1000 fraudapp && chown -R fraudapp:fraudapp /app
USER fraudapp

# Copy compiled packages from builder
COPY --from=builder /install /usr/local

# Copy application code
COPY --chown=fraudapp:fraudapp inference/ ./inference/
COPY --chown=fraudapp:fraudapp model/artifacts/ ./model/artifacts/

# Health check endpoint
HEALTHCHECK --interval=10s --timeout=3s --start-period=15s \
    CMD curl -f http://localhost:8000/health || exit 1

EXPOSE 8000

CMD ["uvicorn", "inference.main:app",
    "--host", "0.0.0.0",
    "--port", "8000",
    "--workers", "4",           # Match CPU cores
    "--loop", "uvloop",        # ~20% faster than asyncio default
    "--access-log"]
```

■ WEEK 11 — CI/CD & GitHub Actions

4.2 CI/CD Pipeline with GitHub Actions

Every commit should trigger automated quality gates. A **30 LPA** role expects you to understand what CI/CD means in practice — not just theory. Here is a production-grade pipeline with three stages: lint/test, build, deploy.

GitHub Actions CI Pipeline — Quality Gate & Integration Test:

YAML

```
# .github/workflows/ci.yml
name: CI - Lint, Test & Build

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  quality-gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: 'pip'

      - name: Install dependencies
        run: pip install -r requirements.txt -r requirements-dev.txt

      - name: Lint with Ruff
        run: ruff check . --output-format=github

      - name: Format check with Black
        run: black --check .

      - name: Run unit tests with coverage
        run: |
          pytest tests/unit/ -v \
            --cov=inference \
            --cov=model \
            --cov-report=xml \
            --cov-fail-under=80

      - name: Upload coverage to Codecov
        uses: codecov/codecov-action@v4

  integration-test:
    runs-on: ubuntu-latest
    needs: quality-gate
    services:
      postgres:
        image: timescale/timescaledb:latest-pg16
        env:
          POSTGRES_PASSWORD: test_secret
          POSTGRES_DB: fraud_test
        ports: ['5432:5432']
    steps:
      - uses: actions/checkout@v4
      - name: Run integration tests
        run: pytest tests/integration/ -v --timeout=60
```

GitHub Actions CD Pipeline — Build, Scan & Push:


```
# .github/workflows/ci.yml (continued – build-push job)

build-push:
  runs-on: ubuntu-latest
  needs: integration-test
  if: github.ref == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4

    - name: Build Docker image
      run: |
        docker build \
          -f docker/Dockerfile.inference \
          -t fraud-engine:${{ github.sha }} .

    - name: Scan for vulnerabilities (Trivy)
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: fraud-engine:${{ github.sha }}
        severity: 'CRITICAL,HIGH'
        exit-code: '1' # Fail CI if critical CVEs found

    - name: Login to GitHub Container Registry
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${ github.actor }
        password: ${ secrets.GITHUB_TOKEN }

    - name: Push image
      run: |
        docker tag fraud-engine:${{ github.sha }} \
          ghcr.io/${ github.repository }/fraud-engine:latest
        docker push ghcr.io/${ github.repository }/fraud-engine:latest
```

■ WEEK 12 — Monitoring & Drift Detection

4.3 Prometheus Metrics & Grafana Dashboard

Observability is what separates junior from senior engineers. You've already instrumented the inference service with Prometheus metrics. Now configure Prometheus to scrape them and build Grafana dashboards.

```
# monitoring/prometheus.yml
global:
  scrape_interval: 15s
  evaluation_interval: 15s

rule_files:
  - 'alerts.yml'

scrape_configs:
  - job_name: 'fraud_inference'
    static_configs:
      - targets: ['inference:8000']
        metrics_path: '/metrics'

  - job_name: 'kafka'
    static_configs:
      - targets: ['kafka:9308'] # kafka-exporter sidecar

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']

alerting:
  alertmanagers:
    - static_configs:
        - targets: ['alertmanager:9093']
```

4.4 Automated Model Degradation Alerts

This is the most impressive feature of the entire system. Data drift occurs when the statistical properties of production transactions diverge from training data. When this happens, model performance silently degrades — fraud gets through. We detect this using the Population Stability Index (PSI).

Drift Detector — Part 1: PSI Computation:

```

# processing/drift_detector.py
import numpy as np
import pandas as pd
from prometheus_client import Gauge
import asyncpg, logging

logger = logging.getLogger(__name__)

PSI_GAUGE = Gauge('feature_psi', 'Population Stability Index',
                  ['feature_name'])
MODEL_PERF = Gauge('model_precision_rolling_1h', 'Rolling 1h precision')

def compute_psi(expected: np.ndarray, actual: np.ndarray,
                n_bins: int = 10) -> float:
    """
    Population Stability Index (PSI):
    PSI < 0.1 -> No significant change (stable)
    PSI 0.1-0.2 -> Moderate drift (investigate)
    PSI > 0.2 -> Major shift (retrain model!)
    """
    _, bin_edges = np.histogram(expected, bins=n_bins)
    bin_edges[0] = -np.inf
    bin_edges[-1] = np.inf

    expected_pct = np.histogram(expected, bins=bin_edges)[0] / len(expected)
    actual_pct = np.histogram(actual, bins=bin_edges)[0] / len(actual)

    # Avoid log(0) - replace zeros with small epsilon
    expected_pct = np.where(expected_pct == 0, 1e-6, expected_pct)
    actual_pct = np.where(actual_pct == 0, 1e-6, actual_pct)

    psi = np.sum((actual_pct - expected_pct)
                 * np.log(actual_pct / expected_pct))
    return float(psi)

```

Drift Detector — Part 2: Async Drift Check & Alerting:

```
# processing/drift_detector.py (continued)

async def run_drift_check(db_pool: asyncpg.Pool,
                        training_stats: dict) -> dict:
    """
    Fetch last 1h of production data, compute PSI per feature,
    update Prometheus gauges. Called by Spark every 30 minutes.
    """
    async with db_pool.acquire() as conn:
        rows = await conn.fetch(
            '''SELECT amount, txn_count_10m, velocity_ratio
               FROM transactions
               WHERE timestamp >= NOW() - INTERVAL '1 hour'
               LIMIT 10000'''
        )

    if not rows:
        logger.warning('No production data for drift check')
        return {}

    prod_df = pd.DataFrame(rows,
                          columns=['amount', 'txn_count_10m', 'velocity_ratio'])
    drift_report = {}

    for feature in ['amount', 'txn_count_10m', 'velocity_ratio']:
        psi = compute_psi(
            expected = training_stats[feature], # Saved at training time
            actual   = prod_df[feature].dropna().values
        )
        PSI_GAUGE.labels(feature_name=feature).set(psi)
        drift_report[feature] = psi

        if psi > 0.2:
            logger.critical(
                f'DATA DRIFT ALERT: {feature} PSI={psi:.3f} > 0.2. '
                f'Model retraining required immediately!'
            )

    return drift_report
```

Grafana Alert Rule — Auto-Trigger Retraining:

```
# monitoring/alerts.yml (Prometheus Alert Rules)
groups:
- name: fraud_model_health
  rules:

    - alert: HighInferenceLatency
      expr: histogram_quantile(0.99, inference_latency_ms_bucket) > 20
      for: 2m
      labels:
        severity: warning
      annotations:
        summary: 'P99 inference latency > 20ms SLA'
        description: 'Check feature store query performance and ONNX runtime.'

    - alert: DataDriftDetected
      expr: feature_psi > 0.2
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: 'Critical data drift detected in feature {{ $labels.feature_name }}'
        description: 'PSI={{ $value }}. Trigger model retraining pipeline.'

    - alert: FraudRateSpike
      expr: rate(fraud_alerts_total[5m]) / rate(requests_total[5m]) > 0.05
      for: 3m
      labels:
        severity: critical
      annotations:
        summary: 'Fraud rate > 5% in last 5 minutes – possible attack in progress'
```

4.5 Interview Preparation & Portfolio Strategy

After completing this project, here is how to position it for maximum impact in ■30 LPA interviews at companies like Razorpay, CRED, Zepto, Flipkart, and PhonePe:

Q: 'Walk me through your system architecture.'

Start with the business problem (fraud costs, latency SLAs), then describe each layer Kafka → Spark → TimescaleDB → FastAPI — with the specific design decisions. Always quantify: '10K TPS ingestion', 'sub-20ms P99', '0.3% fraud rate'.

Q: 'Why did you choose XGBoost over a neural network?'

Neural networks require more data to outperform gradient boosting on tabular data. XGBoost is interpretable via SHAP (regulatory requirement in fintech). It exports cleanly to ONNX. For tabular fraud data with < 1M rows, XGBoost wins.

Q: 'How do you handle model degradation in production?'

Describe your PSI-based drift detection: Spark computes PSI every 30 minutes against training distribution statistics. Prometheus gauge exceeds 0.2 threshold, Grafana alert fires, triggers automated retraining GitHub Actions workflow.

Q: 'What is a Window Function and when would you use one?'

A window function performs calculations across a set of rows related to the current row, without collapsing them like GROUP BY. In fraud detection, I use RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW to count transactions in a rolling 10-minute window per user, which is a key velocity feature.

■ MASTERY CHALLENGE #4 — The Final Boss

Conduct a full end-to-end **load test under simulated attack**: 1. Start all services via docker-compose up. 2. Run the Kafka producer at 2000 TPS for 10 minutes. 3. Inject a card-testing attack pattern (50+ small transactions, one user) at minute 5. 4. Verify the inference service flags the attack (fraud_alerts_total counter spikes). 5. Verify P99 latency stays below 20ms throughout. 6. Screenshot the Grafana dashboard showing the attack and add to your README. This is your portfolio centerpiece. Record a 3-minute Loom demo walking through the architecture.

APPENDICES

Appendix A: Essential Documentation Links

Technology	URL	Key Reference Area
Apache Kafka	kafka.apache.org/documentation	Core concepts, producer/consumer configs, replication
Confluent Python	docs.confluent.io/kafka-clients/python	confluent-kafka Python client API reference
PySpark	spark.apache.org/docs/latest/api/python	Structured Streaming, DataFrame API
Spark + Kafka	spark.apache.org/docs/latest/structured-streaming-kafka-source-sinks.html	Kafka source/sink options, watermarks
TimescaleDB	docs.timescale.com	Hypertables, continuous aggregates, compression
PostgreSQL Window	postgresql.org/docs/current/tutorial-window.html	Window functions reference — must read
FastAPI	fastapi.tiangolo.com	Async endpoints, dependency injection, lifespan
asyncpg	magicstack.github.io/asyncpg	Async PostgreSQL driver for Python
XGBoost	xgboost.readthedocs.io	Parameter tuning, sklearn API, ONNX export
ONNX Runtime	onnxruntime.ai/docs	Inference sessions, quantization, providers
Prometheus Python	prometheus.github.io/client_python	Counter, Gauge, Histogram, Summary
Grafana	grafana.com/docs/grafana/latest	Dashboard creation, alerting, provisioning
Docker Compose	docs.docker.com/compose	Multi-container apps, networking, volumes
GitHub Actions	docs.github.com/en/actions	Workflow syntax, caching, secrets management
scikit-learn	scikit-learn.org/stable/modules/classes.html	Preprocessing, metrics, model selection
imbalanced-learn	imbalanced-learn.org/stable	SMOTE, ADASYN, RandomOverSampler
SHAP	shap.readthedocs.io	XGBoost SHAP values, waterfall plots
Locust	docs.locust.io	Load testing, custom users, latency percentiles

Appendix B: Recommended Learning Resources

- **Kafka: The Definitive Guide (2nd Ed.)** — Narkhede, Shapira, Palino — Free PDF from Confluent (kafka.apache.org/books-and-papers)
- **Designing Data-Intensive Applications** — Martin Kleppmann — The systems design bible (dataintensive.net)
- **Fraud Detection Handbook** — Le Borgne, Bontempi — Free, academic but practical (fraud-detection-handbook.github.io)
- **Fast API Full Course** — Bitfumes on YouTube — Covers async, dependency injection (YouTube: 'FastAPI Full Course')
- **TimescaleDB Tutorial Series** — Official docs — Time-series from scratch (docs.timescale.com/tutorials)

- **XGBoost Paper (Chen & Guestrin)** — The original paper — understand the math (arxiv.org/abs/1603.02754)
- **MLOps Zoomcamp (DataTalks.Club)** — Free 9-week course — covers everything in Phase 4 (github.com/DataTalks-Club/mlops-zoomcamp)
- **SQL Window Functions (Mode Analytics)** — Interactive tutorial with exercises (mode.com/sql-tutorial/sql-window-functions)

Appendix C: Common Debugging Patterns

Symptom	Root Cause	Fix
Kafka consumer lag keeps growing	Processing slower than producing; too few partitions	Increase partitions & consumer group instances
TimescaleDB Seq Scan on large tables	Missing composite index on (user_id, timestamp)	CREATE INDEX idx_txn_user_time ON transactions
FastAPI P99 > 20ms despite ONNX	Synchronous DB calls blocking event loop	Ensure asyncpg (not psycopg2) + await
SMOTE raising ValueError on fit	Class imbalance so extreme that k-NN can't find neighbors	Reduce k_neighbors=3 or use Random Forest
Spark watermark not dropping late data	Watermark threshold too large; state growing unbounded	Reduce watermark; add stateful timeout
Docker compose services fail to connect	Services starting before dependencies are healthy	Add healthcheck + depends_on: condition
ONNX export shape mismatch	Feature count at export != feature count at inference	Use _FEATURES constant and assert
Grafana showing 'No Data'	Prometheus not scraping; wrong port or path	Check prometheus targets UI at :9090/targets

12-Week Timeline Summary

Week	Phase	Primary Deliverable	Key Skill Unlocked
1	1 — Foundations	Docker Compose stack running locally	Docker, environment setup
2	1 — Foundations	Transaction simulator + TimescaleDB schema	Python data generation, SQL DDL
3	2 — Data Engine	Kafka producer streaming 500 TPS	Distributed messaging, partitioning
4	2 — Data Engine	PySpark reads Kafka, writes to TSDB	Streaming architecture, micro-batch
5	2 — Data Engine	Velocity SQL CTEs with window functions	Advanced SQL, TimescaleDB aggregations
6	2 — Data Engine	Late data watermark handling	Streaming correctness guarantees
7	3 — The Brain	XGBoost trained, 5-fold CV AUPRC > 0.85	Imbalanced learning, gradient boost
8	3 — The Brain	ONNX export, quantization < 5ms inference	Model optimization, hardware acceleration
9	3 — The Brain	FastAPI /v1/score endpoint < 20ms P99	Async Python, connection pooling
10	4 — Production	Multi-stage Docker images, docker-compose	Container best practices, security
11	4 — Production	GitHub Actions CI/CD pipeline passing	DevOps, automated testing, IaC
12	4 — Production	Grafana dashboard + PSI drift alerts live	MLOps, observability, drift detection

PHASE 5

UPGRADE 1 — REAL DATA

Replace simulated data with IEEE-CIS Kaggle dataset (590K rows)

■ WHY THIS MATTERS

Every candidate has a fraud detection project with simulated data. Interviewers can tell instantly. Real data has messy distributions, missing values, duplicate transaction IDs, timezone chaos, and class imbalance that is not cleanly 0.3%. Using IEEE-CIS signals that you have worked with production-like data.

Dataset: IEEE-CIS Fraud Detection (Kaggle)

This dataset was released by Vesta Corporation (a real payment gateway) for a Kaggle competition in 2019. It contains 590,540 transactions with 433 features across two files: transaction data and identity data. Fraud rate is 3.5% — realistic for card-not-present e-commerce. Download from: kaggle.com/c/ieee-fraud-detection/data

What You Will Encounter That Simulators Cannot Replicate:

Challenge	Description	What You Learn
V-columns (V1–V339)	Anonymized Vesta engineered features — no documentation	Feature selection under uncertainty, variance inflation
Missing values (up to 90%)	Some columns are 90% NaN — drop or impute strategically	Missing data patterns, MCAR vs MAR vs MNAR
TransactionAmt distribution	Heavy right tail: 99th percentile is \$2,000+ but mean is \$106	Log transforms, outlier treatment, robust statistics
card1–card6 encoding	Hashed card identifiers — must engineer cross-features	Entity encoding, frequency encoding, target encoding
DeviceInfo field	Raw strings: 'Samsung SM-G950U Build/R16NW' — string parsing	String parsing, engineering, regex, NLP basics
Time delta drift	TransactionDT is seconds from a hidden reference time	Relative feature engineering without absolute timestamps
Train/test leakage risk	isFraud label correlated with some V-columns via data collection	Temporal leakage detection, proper cross-validation

IEEE-CIS Data Loading & Initial Cleaning Pipeline:

```
# data/loaders/ieee_cis_loader.py
import pandas as pd
import numpy as np
from pathlib import Path

def load_ieee_cis(data_dir: str) -> pd.DataFrame:
    '''Load and merge IEEE-CIS transaction + identity files.'''
    txn = pd.read_csv(f'{data_dir}/train_transaction.csv')
    iden = pd.read_csv(f'{data_dir}/train_identity.csv')

    df = txn.merge(iden, on='TransactionID', how='left')
    print(f'Shape: {df.shape} | Fraud rate: {df.isFraud.mean():.3%}')
    return df

def reduce_memory(df: pd.DataFrame) -> pd.DataFrame:
    '''Downcast dtypes to reduce RAM from ~1.8GB to ~600MB.'''
    for col in df.select_dtypes('float64').columns:
        df[col] = df[col].astype('float32')
    for col in df.select_dtypes('int64').columns:
        df[col] = df[col].astype('int32')
    return df

def drop_high_null_cols(df: pd.DataFrame,
                        threshold: float = 0.9) -> pd.DataFrame:
    '''Drop columns where > 90% values are missing.'''
    null_rates = df.isnull().mean()
    drop_cols = null_rates[null_rates > threshold].index.tolist()
    print(f'Dropping {len(drop_cols)} columns with >{threshold:.0%} nulls')
    return df.drop(columns=drop_cols)

def engineer_card_features(df: pd.DataFrame) -> pd.DataFrame:
    '''Frequency-encode card identifiers as fraud risk proxies.'''
    for col in ['card1', 'card2', 'card3', 'card5']:
        freq = df[col].value_counts(normalize=True)
        df[f'{col}_freq'] = df[col].map(freq)
    # Cross-feature: card1 + addr1 identifies unique cards more precisely
    df['uid'] = df['card1'].astype(str) + '_' + df['addr1'].astype(str)
    uid_freq = df['uid'].value_counts(normalize=True)
    df['uid_freq'] = df['uid'].map(uid_freq)
    return df

def engineer_time_features(df: pd.DataFrame) -> pd.DataFrame:
    '''TransactionDT is relative seconds – extract cyclical features.'''
    # Modulo to extract day-of-week and hour-of-day approximations
    df['hour'] = (df['TransactionDT'] // 3600) % 24
    df['day_of_week'] = (df['TransactionDT'] // (3600 * 24)) % 7
    df['is_weekend'] = (df['day_of_week'] >= 5).astype(int)
    df['is_night'] = ((df['hour'] >= 22) | (df['hour'] <= 6)).astype(int)
    return df
```

What to Document in Your README After Using Real Data:

- Baseline XGBoost AUPRC on simulated data vs IEEE-CIS — show the drop and explain why
- Top 20 features by SHAP importance — V-columns will dominate; explain what that means
- Precision-Recall curve at multiple thresholds — pick a business-justified threshold
- Memory optimization: before vs after downcast (1.8GB → 600MB = you understand production constraints)
- Leakage check: any feature with > 0.95 correlation to isFraud is a data leak — document how you caught it

Why Fraud is a Graph Problem

Isolated transaction features miss the most profitable fraud patterns. A card tester may have a low-value, innocuous-looking transaction. But if that card shares a `device_id` with 40 other cards that have been flagged, the graph reveals the ring immediately. Three fraud archetypes that only graphs catch:

Fraud Type	Graph Signal	Example
Card Testing Ring	One <code>device_id</code> connected to 50+ <code>card1</code> values	Attacker tests stolen cards from one device
Mule Account Network	Chains of P2P transfers: A->B->C->D->cash-out	Money laundering via layered transfers
Bust-Out Fraud	Shared address/phone across newly opened accounts	Synthetic identity fraud cluster
Account Takeover	New <code>device_id</code> on existing <code>user_id</code> with high-value	Credential-stuffed account used immediately

Practical Implementation: NetworkX Graph Features (No GNN Required)

You do NOT need to implement a Graph Neural Network to impress interviewers. What you need is to demonstrate you understand the concept and can extract simple graph features using NetworkX that feed into your XGBoost model. This alone separates you from 95% of candidates.

```

# model/graph_features.py
import networkx as nx
import pandas as pd
import numpy as np

def build_fraud_graph(df: pd.DataFrame) -> nx.Graph:
    """
    Build bipartite graph: card nodes <-> device nodes
    Edge = a transaction occurred between card and device.
    High-degree device nodes = card testing rings.
    """
    G = nx.Graph()
    for _, row in df.iterrows():
        card_node = f"card_{row['card1']}"
        device_node = f"device_{row['DeviceInfo']}"
        G.add_node(card_node, node_type='card')
        G.add_node(device_node, node_type='device')
        G.add_edge(card_node, device_node,
                    amount=row['TransactionAmt'],
                    is_fraud=row['isFraud'])

    return G

def extract_graph_features(df: pd.DataFrame,
                           G: nx.Graph) -> pd.DataFrame:
    """
    For each transaction, compute graph-based risk signals.
    These become new columns in your feature matrix.
    """
    graph_feats = []

    for _, row in df.iterrows():
        card_node = f"card_{row['card1']}"
        device_node = f"device_{row['DeviceInfo']}"

        # How many cards share this device? (ring signal)
        device_degree = G.degree(device_node) if device_node in G else 0

        # How many devices has this card been used on? (takeover signal)
        card_degree = G.degree(card_node) if card_node in G else 0

        # What % of device's connected cards are fraudulent?
        if device_node in G and G.degree(device_node) > 0:
            neighbors = list(G.neighbors(device_node))
            fraud_nbrs = sum(
                1 for n in neighbors
                if any(G[device_node][n].get('is_fraud', 0)
                    for _ in [None])
            )
            device_fraud_rate = fraud_nbrs / len(neighbors)
        else:
            device_fraud_rate = 0.0

        graph_feats.append({
            'device_degree': device_degree,
            'card_degree': card_degree,
            'device_fraud_rate': device_fraud_rate,
            'is_shared_device': int(device_degree > 3),
            'is_card_ring_member': int(device_degree > 10),
        })

    return pd.DataFrame(graph_feats, index=df.index)

```

■ INTERVIEW TALKING POINT

When asked about graph fraud detection say: 'I implemented a bipartite card-device graph using NetworkX to extract degree centrality and neighborhood fraud rate features. These graph features improved my XGBoost AUPRC from 0.87 to 0.91 because they capture ring structures invisible to row-level features. The production path for this at scale is GraphSAGE or a specialized graph DB like Neo4j, but NetworkX was sufficient to validate the signal.' That answer alone is worth ■3-5 LPA in offer differential.

What to Add to Your README:

- A graph visualization (use matplotlib + networkx) showing one confirmed fraud ring — include this as a PNG in your repo
- SHAP importance comparison: model without graph features vs with — show the delta in AUPRC
- One paragraph explaining why you chose NetworkX over Neo4j for this project (cost, complexity, scale threshold)
- Note that at 10M+ transactions/day, graph computation would move to Apache Spark GraphX or a dedicated graph DB

Why Explainability is Non-Negotiable in Indian Fintech

RBI's guidelines on algorithmic credit decisions and fraud flags require that any automated adverse action (transaction decline, account freeze) must be explainable to the affected customer. CRED, Razorpay, and PhonePe all have dedicated explainability layers in their fraud systems. If you have one and the next candidate doesn't, you win the offer. It is that simple.

SHAP Integration into the Training Pipeline:

PYTHON

```
# model/explain.py
import shap
import xgboost as xgb
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

def compute_shap_values(model: xgb.XGBClassifier,
                        X: pd.DataFrame) -> shap.Explanation:
    '''Compute SHAP values using TreeExplainer (fast for XGBoost).'''
    explainer = shap.TreeExplainer(model)
    shap_values = explainer(X) # Returns Explanation object
    return shap_values

def plot_shap_summary(shap_values, X: pd.DataFrame,
                     output_path: str = 'reports/shap_summary.png'):
    '''Generate beeswarm plot – include this in your README.'''
    plt.figure(figsize=(10, 8))
    shap.summary_plot(shap_values, X, show=False)
    plt.tight_layout()
    plt.savefig(output_path, dpi=150, bbox_inches='tight')
    plt.close()
    print(f'SHAP summary plot saved to {output_path}')

def get_top_reasons(shap_values_row: np.ndarray,
                    feature_names: list,
                    n: int = 3) -> list[dict]:
    '''
    For a single prediction, return top-N features driving the score.
    Used by the /v1/explain API endpoint.
    '''
    pairs = list(zip(feature_names, shap_values_row))
    # Sort by absolute SHAP value – biggest impact first
    pairs.sort(key=lambda x: abs(x[1]), reverse=True)
    return [
        {
            'feature': name,
            'shap_value': round(float(val), 4),
            'direction': 'increases_fraud_risk' if val > 0
                        else 'decreases_fraud_risk',
        }
        for name, val in pairs[:n]
    ]
```

Adding /v1/explain to the FastAPI Service:

```
# inference/main.py – add this endpoint alongside /v1/score

from .explain import get_top_reasons
import shap

# In lifespan startup, also load SHAP explainer:
# app.state.explainer = shap.TreeExplainer(xgb_model)

@app.post('/v1/explain')
async def explain_transaction(request: TransactionRequest):
    """
    Returns fraud score + top 3 feature contributions.
    Designed for: customer support teams, compliance audits,
    and RBI reporting on automated adverse actions.
    """
    features = await app.state.extractor.get_features(
        user_id=request.user_id, amount=request.amount,
        merchant_cat=request.merchant_cat, country=request.country,
        timestamp=request.timestamp
    )
    fraud_prob = app.state.predictor.predict(features)
    shap_vals = app.state.explainer.shap_values(features.vector)[0]
    top_reasons = get_top_reasons(shap_vals, features.feature_names, n=3)

    return {
        'transaction_id': request.transaction_id,
        'fraud_probability': round(fraud_prob, 4),
        'risk_level': 'HIGH' if fraud_prob > 0.7 else
                     'MEDIUM' if fraud_prob > 0.3 else 'LOW',
        'explanation': {
            'summary': f'Score driven primarily by {top_reasons[0]["feature"]}',
            'top_factors': top_reasons,
            'model_version': '1.0.0',
            'explainability_method': 'SHAP TreeExplainer',
        }
    }

# Example response for a high-risk transaction:
# {
#   'fraud_probability': 0.923,
#   'risk_level': 'HIGH',
#   'explanation': {
#     'top_factors': [
#       {'feature': 'velocity_ratio', 'shap_value': 1.84,
#        'direction': 'increases_fraud_risk'},
#       {'feature': 'unique_devices_10m', 'shap_value': 0.91,
#        'direction': 'increases_fraud_risk'},
#       {'feature': 'amount_zscore', 'shap_value': 0.67,
#        'direction': 'increases_fraud_risk'}
#     ]
#   }
# }
```

■ README SECTION TO ADD

Add a section called 'Model Explainability & Compliance'. Include: (1) The SHAP beeswarm plot PNG. (2) A sample /v1/explain API response. (3) One paragraph stating: 'This endpoint supports compliance with RBI guidelines on algorithmic decision transparency by providing human-readable feature attributions for every automated fraud decision.' This sentence in your README is worth more than 200 lines of code in an interview.

UPGRADE 4 — AUTOMATED RETRAINING

Close the MLOps loop: drift detected -> retrain -> A/B test -> promote

The MLOps Loop — What Most Projects Are Missing

Drift detection without automated response is just expensive logging. A complete production ML system has a closed feedback loop: monitor → detect → retrain → validate → promote. This loop is what separates a ■22 LPA 'I built a model' profile from a ■30 LPA 'I operate ML systems' profile.

Stage	Trigger	Action	Success Criteria
Monitor	Continuous (every 30 min)	Compute PSI per feature via Spark	PSI < 0.1 all features
Detect	PSI > 0.2 on any feature	Fire Prometheus alert + GitHub Action	Alert delivered < 2 min
Retrain	Alert received	Re-run training on rolling 90-day window	Job completes < 30 min
Validate	Retrain complete	Compare new vs current AUPRC on holdout	New AUPRC >= current - 0.01
A/B Test	Validation passes	Route 10% traffic to new model (canary)	No latency regression, no AUPRC
Promote	A/B passes 24h	Swap models, update Prometheus model_version	Zero downtime swap
Rollback	A/B fails or error spike	Revert to previous model automatically	Rollback < 60 seconds

GitHub Actions Retraining Workflow:


```
# .github/workflows/retrain.yml
name: Automated Model Retraining

on:
  # Manual trigger from Grafana alert webhook
  workflow_dispatch:
    inputs:
      trigger_reason:
        description: 'Why retraining was triggered'
        required: true
        default: 'data_drift_detected'
  # Also run every Sunday night as scheduled retraining
  schedule:
    - cron: '0 2 * * 0' # 02:00 UTC every Sunday

jobs:
  retrain:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with: {python-version: '3.11', cache: 'pip'}

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Export training data (rolling 90-day window)
        env:
          DB_URL: ${ secrets.TIMESCALEDDB_URL }
        run: python model/export_training_data.py --window-days=90

      - name: Train new model
        run: python model/train.py --output=model/artifacts/candidate.pkl

      - name: Evaluate against holdout
        run: |
          python model/evaluate.py \
            --model=model/artifacts/candidate.pkl \
            --baseline=model/artifacts/production.pkl \
            --output=reports/retrain_comparison.json

      - name: Check if new model is better
        id: check_improvement
        run: |
          python -c "
          import json
          r = json.load(open('reports/retrain_comparison.json'))
          ok = r['candidate_auprc'] >= r['baseline_auprc'] - 0.01
          print(f'Candidate AUPRC: {r[\"candidate_auprc\"]:.4f}')
          print(f'Baseline AUPRC: {r[\"baseline_auprc\"]:.4f}')
          exit(0 if ok else 1) # Fail job if regression detected
          "

      - name: Export to ONNX + quantize
        if: steps.check_improvement.outcome == 'success'
        run: python model/export_onnx.py --input=model/artifacts/candidate.pkl

      - name: Build and push new inference image
        if: steps.check_improvement.outcome == 'success'
        run: |
          docker build -f docker/Dockerfile.inference \
            -t fraud-engine:retrain-${ github.run_id } .
          # Push to registry and trigger canary deployment
```

Canary Deployment with Docker Compose:

```

# For local / small-scale: use Docker Compose with two inference services
# inference_v1 handles 90% traffic, inference_v2 handles 10% (canary)

# In docker-compose.yml, add alongside existing inference service:

# inference_canary:
#   build: {context: ., dockerfile: docker/Dockerfile.inference}
#   environment:
#     - MODEL_PATH=model/artifacts/candidate_quantized.onnx
#     - MODEL_VERSION=candidate
#   ports: ['8001:8000'] # Different port

# In your Kafka consumer / load balancer, route 10% of transactions
# to port 8001 and compare fraud_score distributions in Grafana.

# model/evaluate.py – the key comparison function
import json, joblib, numpy as np
from sklearn.metrics import average_precision_score
import pandas as pd

def compare_models(candidate_path: str, baseline_path: str,
                   test_data_path: str, output_path: str):
    candidate = joblib.load(candidate_path)
    baseline = joblib.load(baseline_path)
    df = pd.read_parquet(test_data_path)

    from model.train import FEATURE_COLS, engineer_features
    df = engineer_features(df)
    X, y = df[FEATURE_COLS].fillna(0), df['isFraud']

    cand_auprc = average_precision_score(y, candidate.predict_proba(X)[:,:1])
    base_auprc = average_precision_score(y, baseline.predict_proba(X)[:,:1])

    result = {'candidate_auprc': cand_auprc, 'baseline_auprc': base_auprc,
              'delta': cand_auprc - base_auprc,
              'promote': cand_auprc >= base_auprc - 0.01}
    with open(output_path, 'w') as f:
        json.dump(result, f, indent=2)
    print(json.dumps(result, indent=2))
    return result

```

■ THE BRUTAL TRUTH

Most candidates who build this project treat it as copy-paste engineering. Senior interviewers spend exactly 5 minutes on 'tell me about your project' and 45 minutes on 'explain X at a deeper level'. The math in this section is what gets you from ■22 LPA to ■30 LPA. You do not need to memorize equations — you need to understand what they mean and be able to sketch them on a whiteboard.

1. Gradient Boosting — What `scale_pos_weight` Actually Does

XGBoost minimizes a loss function via gradient descent in function space. For binary classification, the default loss is log-loss (binary cross-entropy). The `scale_pos_weight` parameter multiplies the gradient and hessian contributions of positive (fraud) samples by that factor. Here is what this means:

PYTHON

```
# MATHEMATICAL INTUITION (no code – read and understand this):

# Standard log-loss gradient for one sample:
#   g_i = pred_i - label_i   (first derivative)
#   h_i = pred_i * (1 - pred_i) (second derivative / hessian)

# With scale_pos_weight=333 (for 0.3% fraud rate):
#   For fraud samples (label=1): g_i *= 333, h_i *= 333
#   For legit samples (label=0): g_i *= 1, h_i *= 1

# Effect: The model sees each fraud sample as if it appeared 333 times.
# This forces the tree splits to prioritize fraud separation.

# WHY AUPRC IS THE RIGHT METRIC (not AUC-ROC):
# AUC-ROC measures discrimination across ALL thresholds.
# At 0.3% fraud rate, a model that flags everything as fraud
# still gets AUC-ROC > 0.5 because it separates classes.
# AUPRC (Area Under Precision-Recall Curve) punishes the model
# for high false positive rate on the majority (legit) class.
# In fraud: FP = wrongly declined legit customer = lost revenue.
# AUPRC directly measures the trade-off you care about.

# INTERVIEW QUESTION: 'Why not just use accuracy?'
# ANSWER: 'With 0.3% fraud rate, a model predicting all-legitimate
# achieves 99.7% accuracy but catches zero fraud. Accuracy is
# meaningless for severely imbalanced problems. AUPRC at our
# operating threshold of 0.15 is the right business metric.'
```

2. PSI vs KL-Divergence — The Difference Interviewers Probe

Both PSI and KL-divergence measure distribution shift. Interviewers at senior levels will ask you to distinguish them. Here is the answer:

Property	PSI	KL-Divergence
Formula	$\text{SUM}((P-Q) * \ln(P/Q))$	$\text{SUM}(P * \ln(P/Q))$
Symmetric?	YES — $\text{PSI}(P,Q) == \text{PSI}(Q,P)$	NO — $\text{KL}(P,Q) \neq \text{KL}(Q,P)$
Interpretable thresholds?	YES — 0.1/0.2/0.25 industry standard	NO — no standard thresholds

Property	PSI	KL-Divergence
Handles zero bins?	With epsilon smoothing, yes	Undefined if Q=0
When to use	Production drift monitoring (symmetric, interpretable)	Model training (loss functions, VAEs)
Drift threshold	< 0.1: stable, 0.1-0.2: watch, > 0.2: retrain	No standard — relative comparison only

3. SQL Window Function Execution — What EXPLAIN ANALYZE Tells You

When you run EXPLAIN ANALYZE on your velocity query, you need to understand every line. This is asked in data engineering interviews at L4+ levels. Here are the four terms you must know cold:

SQL

```

-- Run this on your TimescaleDB after loading data:
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT user_id, COUNT(*) OVER (
  PARTITION BY user_id ORDER BY timestamp
  RANGE BETWEEN INTERVAL '10 minutes' PRECEDING AND CURRENT ROW
) AS txn_count_10m
FROM transactions
WHERE user_id = 'user_00001'
  AND timestamp >= NOW() - INTERVAL '1 hour';

-- WHAT TO LOOK FOR IN THE OUTPUT:

-- 1. 'Index Scan using idx_txn_user_time' -> GOOD
--    'Seq Scan on transactions' -> BAD (missing index, fix immediately)

-- 2. 'Rows Removed by Filter: 0' -> index is selective
--    'Rows Removed by Filter: 99000' -> index not helping, investigate

-- 3. 'WindowAgg' node -> this is the window function execution
--    cost=X..Y: X=startup cost, Y=total cost. Lower is better.

-- 4. 'Buffers: shared hit=N' -> N pages read from RAM cache (fast)
--    'Buffers: shared read=N' -> N pages read from disk (slow)
--    If 'read' >> 'hit': your working set doesn't fit in shared_buffers.
--    Fix: increase shared_buffers in postgresql.conf to 25% of RAM.

-- TARGET: For a 10M-row hypertable, your velocity query should:
--   Execute in < 10ms (index scan, not seq scan)
--   Show 'Buffers: shared hit' >> 'shared read' (hot cache)
--   Show chunk exclusion: TimescaleDB only scans relevant time chunks

```

4. Kafka Internals — Three Concepts That Trip Up Interviews

Concept	What It Is	Why It Matters for Fraud
Log Compaction	Kafka retains only the latest value per key; older entries are deleted	Keeps latest user state without infinite log growth
Consumer Group Rebalance	Partitions reassigned when a consumer joins or dies	Brief pause in fraud processing; understand idempotence
Exactly-Once Semantics	Transactional producers + idempotent consumers = Double-Check	Duplicate transaction processing = double fraud
Partition Key Choice	We key by user_id — all user txns go to same partition	Enables in-order per-user processing; velocity
Retention vs Replay	Default 7-day retention; rewind offsets to replay for retraining	Replay last 90 days of transactions to retrain models

Frame Your Model in Business Terms

The difference between a ■22 LPA answer and a ■30 LPA answer in a final-round interview is almost always this: the senior candidate quantifies business impact. Use this framework to compute a number you can state confidently.

PYTHON

```
# Business impact calculator – put this in a Jupyter notebook
# and include the output in your README

# === ASSUMPTIONS (use IEEE-CIS dataset statistics) ===
total_daily_transactions = 50_000      # Hypothetical production volume
fraud_rate               = 0.035       # 3.5% per IEEE-CIS
avg_fraud_amount_inr     = 8_500       # ~$100 USD equivalent
avg_txn_amount_inr       = 1_200       # Average transaction

# === MODEL PERFORMANCE AT CHOSEN THRESHOLD (0.15) ===
precision = 0.72    # Of all flagged: 72% are actual fraud
recall    = 0.81    # Of all fraud: 81% are caught

# === DAILY CALCULATIONS ===
daily_frauds      = total_daily_transactions * fraud_rate
caught_fraud_count = daily_frauds * recall
false_positive_count = (caught_fraud_count / precision) - caught_fraud_count

# Financial impact
fraud_prevented_inr = caught_fraud_count * avg_fraud_amount_inr
fp_friction_cost_inr = false_positive_count * avg_txn_amount_inr * 0.02
net_daily_benefit_inr = fraud_prevented_inr - fp_friction_cost_inr

print(f'Daily frauds in system:      {daily_frauds:.0f}')
print(f'Fraud caught (recall=0.81):  {caught_fraud_count:.0f}')
print(f'False positives (legit declines): {false_positive_count:.0f}')
print(f'Fraud prevented (INR):        ■{fraud_prevented_inr:,.0f}')
print(f'FP friction cost (INR):       ■{fp_friction_cost_inr:,.0f}')
print(f'Net daily benefit (INR):      ■{net_daily_benefit_inr:,.0f}')
print(f'Annualised benefit (INR):     ■{net_daily_benefit_inr*365:,.0f}')

# SAMPLE OUTPUT:
# Daily frauds in system:      1,750
# Fraud caught (recall=0.81):  1,418
# False positives (legit declines): 551
# Fraud prevented (INR):        ■1,20,53,000
# FP friction cost (INR):       ■13,224
# Net daily benefit (INR):      ■1,20,39,776
# Annualised benefit (INR):     ■43,94,51,624 (~■44 crore/year)
```

■ THE INTERVIEW ANSWER

When asked 'What is the business impact of your model?', say: 'At our operating threshold of 0.15, the model achieves 81% recall and 72% precision. On a hypothetical volume of 50,000 daily transactions at 3.5% fraud rate, this prevents approximately ■1.2 crore in daily fraud losses while generating only 551 false positive declines. The net annual benefit is approximately ■44 crore. The threshold was chosen to maximize net benefit at the business's stated cost ratio of 10:1 between fraud loss and customer friction.' No other fresher in that interview room will say something like this.

Final Portfolio Checklist — Before Applying

Do not send a single application until every item below is ticked. Recruiters spend 90 seconds on a GitHub profile. Make every second count.

■ README.md

- ✓ System architecture diagram (draw.io or Excalidraw, export as PNG)
- ✓ One-command local setup: docker-compose up — confirm it actually works on a fresh machine
- ✓ SHAP beeswarm plot image embedded directly in README
- ✓ Business impact table (the ■44 crore calculation above)
- ✓ Latency benchmark screenshot: Locust P99 < 20ms under 200 concurrent users
- ✓ Grafana dashboard screenshot showing live fraud alert firing

■ Code Quality

- ✓ Ruff linter passes with zero errors (add badge to README)
- ✓ pytest coverage > 80% (add Codecov badge to README)
- ✓ All functions have docstrings explaining what, why, and expected input/output
- ✓ No hardcoded credentials — all secrets via environment variables
- ✓ requirements.txt has pinned versions (e.g. xgboost==2.0.3, not xgboost>=2.0)

■ Git Hygiene

- ✓ Meaningful commit messages: 'feat: add SHAP /v1/explain endpoint' not 'update code'
- ✓ Feature branches merged via PRs (shows professional workflow even solo)
- ✓ GitHub Actions CI badge showing green on main branch
- ✓ model/artifacts/*.onnx added to .gitignore (never commit large binary files)

■ Demo Video (3 minutes, Loom)

- ✓ Show docker-compose up bringing the full stack live from scratch
- ✓ Show the Kafka producer streaming transactions in real time
- ✓ Hit /v1/score with a card-testing attack pattern — show HIGH risk response
- ✓ Hit /v1/explain — show SHAP feature attributions in the response
- ✓ Show the Grafana fraud_alerts_total counter spike and PSI gauge
- ✓ Include the Loom link prominently in your README and LinkedIn

Target Companies & Application Strategy

Company	Role Title to Search	Stack Match	Expected Range
Razorpay	ML Engineer — Risk & Fraud	Kafka, Spark, XGBoost, FastAPI	■25–35 LPA
CRED	Data Engineer / ML Engineer	Kafka, dbt, Python, Spark	■28–38 LPA
PhonePe	SDE-2 / ML Engineer — Payments	Kafka, Flink, Spark, ONNX	■25–32 LPA
Zepto	Data Engineer / ML Platform	Kafka, Spark, Airflow, FastAPI	■22–30 LPA
Paytm	ML Engineer — Risk	Spark, XGBoost, Python	■18–28 LPA

Company	Role Title to Search	Stack Match	Expected Range
BrowserStack	Data Engineer	Kafka, Spark, PostgreSQL	■20–28 LPA
Groww	ML Engineer — Risk & Compliance	Python, XGBoost, FastAPI, Docker	■22–30 LPA
MoEngage	Data Engineer	Kafka, Spark, Python	■18–25 LPA

This is the complete, unfiltered roadmap. The original 12 weeks builds the system. These 6 upgrades close the gap to ■30 LPA. The honest sequence is: build Phase 1–4 first (get something working), then layer in the upgrades one by one. Graph features and real data have the highest ROI per hour invested. The retraining pipeline is the most impressive MLOps signal. The math section is what gets you through the technical deep-dive round. You now have everything. The only variable left is execution.

— Staff Machine Learning Engineer | Real-Time Fraud & Financial Intelligence | 2025